Pattern Matching

- Document number: P1308R0
- Date: 2018-10-07

- Reply-to: David Sankel <dsankel@bloomberg.net>, Dan Sarginson<dsarginson@bloomberg.net>, Sergei Murzin<smurzin@bloomberg.net>

- Audience: Evolution

Abstract

Pattern matching improves code readability, makes the language easier to use, and prevent common mishaps.

History

P1308R0. Initial version of this paper which was extracted from P0095R1. Disambiguation between a expression `inspect` and a statement `inspect` is now based on the context in which it appears. Parentheses are now used to disambiguate between `constexpr` expressions and new identifiers. Matching may now be used on inheritance hierarchies. Matching against constexpr expressions has been extended to all types instead of only supporting integral and enum types. Additionally a new Open Design Questions section was added.

Introduction

There is general agreement that pattern matching would make C++ programmers more productive. However, a design that has the elegance of a functional language, but with the performance and utility of a systems language is not forthcoming. Add to that the requirements of backwards compatibility and consistency with the rest of C++, and we've got quite a challenging problem.

This paper presents a design for pattern matching that the authors feel has the right mix of syntactic elegance, low-level performance, and consistency with other language features.

Figure 1: Switching an enum.

before

```
enum color { red, yellow, green, blue };

const Vec3 opengl_color = [&c] {
  switch(c) {
    case red:
      return Vec3(1.0, 0.0, 0.0);
      break;
    case yellow:
      return Vec3(1.0, 1.0, 0.0);
      break;
    case green:
      return Vec3(0.0, 1.0, 0.0);
      break;
    case blue:
      return Vec3(0.0, 0.0, 1.0);
      break;
    default:
```

after

```
const Vec3 opengl_color =
  inspect(c) {
    red    => Vec3(1.0, 0.0, 0.0)
    yellow => Vec3(1.0, 1.0, 0.0)
    green  => Vec3(0.0, 1.0, 0.0)
    blue   => Vec3(0.0, 0.0, 1.0)
  };
```

```
    std::abort();
  }();
```

Figure 2: `struct` inspection

before	after

`struct player {`
`  std::string name;`
`  int hitpoints;`
`  int lives;`
`};`	

`void takeDamage(player &p) {`
`  if(p.hitpoints == 0 &&`
`  p.lives == 0)`
`  gameOver();`
`  else if(p.hitpoints == 0) {`
`  p.hitpoints = 10;`
`  p.lives--;`
`  }`
`  else if(p.hitpoints <= 3) {`
`  p.hitpoints--;`
`  messageAlmostDead();`
`  }`
`  else {`
`  p.hitpoints--;`
`  }`
`}`
	`void takeDamage(player &p) {`
	`  inspect(p) {`
	`  [hitpoints:  0, lives:0]  => gameOver();`
	`  [hitpoints:hp@0, lives:l]  => hp=10, l--;`
	`  [hitpoints:hp] if (hp <= 3) => { hp--;`
	`  messageAlmostDead(); }`
	`  [hitpoints:hp] => hp--;`
	`  }`
	`}`

Motivation

A well-designed pattern matching syntax makes simple problems simple, and enables programmers to express their solution in an intuitive way. The C++ language continuously evolves to allow us to deal with common tasks using constructs that abstract away much of the ‘how’ and allow us to focus on the ‘what’, and in this way complex comparison and selection semantics have not moved with the state of the art.

In the same way that smart pointers have allowed us to express our high-level intent without the risks associated with manual memory management, pattern matching will allow the programmer to clearly express their high-level goal without the tedious and potentially error-prone task of creating and testing deeply-nested conditional constructs.

As proposed, pattern matching is *teachable*: it can be easily explained by one developer to another. Pattern matching is *reviewable*: it can be quickly read and verified to achieve the desired effect. Pattern matching is *powerful*: it can achieve a lot in a a small amount of terse syntax.

Pattern Matching With `inspect`

This section overviews the proposed pattern matching syntax and how it applies to integral types, arrays and non-union classes with public data members. A future paper will explore opt-in extensions for user-defined and STL types.

Pattern matching builds upon structured binding and follows the same name binding rules. The syntax is extended for pattern matching.

Some terms used in this paper:

- We use the word `struct` to mean non-union class with public data members.
- The word "piece" is used to denote a field in such a `struct`.

Pattern matching values

The most basic pattern matching is that of values and that is the subject of this section. Before we get there, however, we need to distinguish between the two places pattern matching can occur. The first is in the statement context. This context is most useful when the intent of the pattern is to produce some kind of action. The `if` statement, for example, is used in this way. The second place pattern matching can occur is is in an expression context. Here the intent of the pattern is to produce a value of some sort. The ternary operator `?:`, for example, is used in this context. Upcoming examples will help clarify the distinction.

In the following example, we're using `inspect` as a *statement* to check for certain values of an int `i`:

```
inspect(i) {
  0 =>
    std::cout << "I can't say I'm positive or negative on this syntax."
              << std::endl;
  6 =>
    std::cout << "Perfect!" << std::endl;
  _ =>
    std::cout << "I don't know what to do with this." << std::endl;
}
```

The _ character is the pattern which always succeeds. It represents a wildcard or fallthrough. The above code is equivalent to the following `switch` statement.

```
switch(i) {
  case 0:
    std::cout << "I can't say I'm positive or negative on this syntax."
              << std::endl;
    break;
  case 6:
    std::cout << "Perfect!" << std::endl;
    break;
  default:
    std::cout << "I don't know what to do with this." << std::endl;
}
```

In the following example, we're using `inspect` as an *expression*, to match a pattern and produce a value based upon `c`, an instance of the `color enum`:

```
enum color { red, yellow, green, blue };

// elsewhere...

const Vec3 opengl_color = inspect(c) {
                            red    => Vec3(1.0, 0.0, 0.0)
                            yellow => Vec3(1.0, 1.0, 0.0)
                            green  => Vec3(0.0, 1.0, 0.0)
                            blue   => Vec3(0.0, 0.0, 1.0)
                          };
```

Note that the cases do not end in a semicolon when appearing in an expression context.

It is also important to note that if an `inspect` expression does not have a matching pattern, an `std::no_match` exception is thrown. This differs from `inspect` statements which simply move on to the next statement if no pattern matches.

All we've seen so far is a condensed and safer `switch` syntax which can also be used in expressions of any type (==-comparison is used). Pattern matching's real power comes when we use more complex patterns. We'll see some of that below.

Pattern matching structs

Pattern matching `structs` in isolation isn't all that interesting: they merely bind new identifiers to each of the fields.

```
struct player {
  std::string name;
  int hitpoints;
  int coins;
};

void log_player( const player & p ) {
  inspect(p) {
    [n,h,c]
      => std::cout << n << " has " << h << " hitpoints and " << c << " coins.";
  }
}
```

`n`, `h`, and `c` are "bound" to their underlying values in a similar way to structured bindings. See structured binding for more information on what it means to bind a value.

`struct` patterns aren't limited to binding new identifiers though. We can instead use a nested pattern as in the following example.

```
void get_hint( const player & p ) {
  inspect( p ) {
    [_, 1, _] => std::cout << "You're almost destroyed. Give up!" << std::endl;
    [_,10,10] => std::cout << "I need the hints from you!" << std::endl;
    [_, _,10] => std::cout << "Get more hitpoints!" << std::endl;
    [_,10, _] => std::cout << "Get more ammo!" << std::endl;
    [n, _, _] => if( n != "The Bruce Dickenson" )
                   std::cout << "Get more hitpoints and ammo!" << std::endl;
                 else
                   std::cout << "More cowbell!" << std::endl;
  }
}
```

While the above code is certainly condensed, it lacks clarity. It is tedious to remember the ordering of a `struct`'s fields. Not all is lost, however. Alternatively we can match using field names.

```
void get_hint( const player & p ) {
  inspect(p) {

    [hitpoints:1]
      => std::cout << "You're almost destroyed. Give up!" << std::endl;

    [hitpoints:10, coins:10]
```

```
      => std::cout << "I need the hints from you!" << std::endl;

    [coins:10]
      => std::cout << "Get more hitpoints!" << std::endl;

    [hitpoints:10]
      => std::cout << "Get more ammo!" << std::endl;

    [name:n]
      => if( n != "The Bruce Dickenson" )
           std::cout << "Get more hitpoints and ammo!" << std::endl;
         else
           std::cout << "More cowbell!" << std::endl;
  }
}
```

Finally, our patterns can incorporate guards through use of an if clause. The last pattern in the above function can be replaced with the following two patterns:

```
[name:n] if( n == "The Bruce Dickenson" ) => std::cout << "More cowbell!" << std::endl;
_ => std::cout << "Get more hitpoints and ammo!" << std::endl;
```

Alternative field matching syntax

With the addition of designated initialization to C++ with P0329, it may be interesting to use similar `.` syntax for matching by field name. With this approach the above example would look as follows:

```
void get_hint( const player & p ) {
  inspect(p) {

    [.hitpoints 1]
      => std::cout << "You're almost destroyed. Give up!" << std::endl;

    [.hitpoints 10, .coins 10]
      => std::cout << "I need the hints from you!" << std::endl;

    [.coins 10]
      => std::cout << "Get more hitpoints!" << std::endl;

    [.hitpoints 10]
      => std::cout << "Get more ammo!" << std::endl;

    [.name n]
      => if( n != "The Bruce Dickenson" )
           std::cout << "Get more hitpoints and ammo!" << std::endl;
         else
           std::cout << "More cowbell!" << std::endl;
  }
}
```

We omit usage of `=` to make it less likely the pattern match will be confused with an assignment.

Pattern matching arrays

Similar to structured binding, we can define pattern matching for fixed-size arrays, by binding all elements in order.

```
const int dimensions[3] = { 1, 4, 2 };

inspect(dimensions) {
  [x, _, _] if (x > 10)
    => std::cout << "First dimension must be less than 10!" << std::endl;
  [x, y, z] if (x + y + z > 100)
    => std::cout << "Sum of dimensions should be less than 100!" << std::endl;
}
```

Similar to struct pattern matching above, we can also allow index-based binding of elements, instead of binding all elements.

Pattern matching pointers

To allow pattern matching over pointer types, we introduce two patterns.

`nullptr` pattern is used to match over a pointer that matches `p == nullptr` and we use `*<pattern>` pattern to match a type that has valid dereference operator and value that it points to matches `<pattern>`.

```
struct node {
  std::unique_ptr<node> left;
  std::unique_ptr<node> right;
  int value;
};

template <typename Visitor>
void visit_leftmost(const node& node, Visitor&& visitor) {
    inspect(node) {
        [left: nullptr] => visitor(node);
        [left: *left_child] => visit_leftmost(left_child,
                                              std::forward<Visitor>(visitor));
    }
}
```

A special dereferencing pattern syntax may seem strange for folks coming from a functional language. However, when we take into account that C++ uses pointers for all recursive structures it makes a lot of sense. Without it, the above pattern would be much more complicated.

Pattern matching class hierarchies

Pattern matching must allow inspection of class hierarchies to extend the improvements in productivity and expressiveness to those working primarily within an object oriented paradigm.

```
class Animal {
public:
  virtual void speak() const = 0;
};

class Cat : public Animal {
public:
  virtual void speak() const override {
    std::cout << "Meow from " << name << std::endl;
  }

  std::string name;
};
```

```
class Crow : public Animal {
public:
  virtual void speak() const override {
    std::cout << "Caaaw!" << std::endl;
  }
};

void listen(const Animal &a) {
  inspect(a) {
    Cat c@["Fluffy"] => fluffySays(c.speak());
    Cat [name] => storeNewCatName(name);
    Crow c => std::cout << "All crows say " << c.speak() << std::endl;
  }
}
```

Order of pattern evaluation

In order to keep code easy to read and understand we propose to keep order of pattern evaluation linear. Evaluation proceeds from the first to the last supplied pattern and the first acceptable pattern that matches causes evaluation to execute the associated expression and terminate the search.

For example, inspecting class hierarchies in this way will select the first acceptable match from the list of potential patterns. There is no concept of a ‘best’ or ‘better’ match.

Wording Skeleton

What follows is an incomplete wording for inspection presented for the sake of discussion.

Inspect Statement

inspect-statement:
  `inspect` `(` *expression* `)` `{` *inspect-statement-cases*$_{opt}$ `}`

inspect-statement-cases:
  inspect-statement-case inspect-statement-cases$_{opt}$

inspect-statement-case:
  guarded-inspect-pattern `=>` *statement*

The identifiers in *inspect-pattern* are available in *statement*.

In the case that none of the patterns match the value, execution continues.

Inspect Expression

inspect-expression:
  `inspect` `(` *expression* `)` `{` *inspect-expression-cases*$_{opt}$ `}`

inspect-expression-cases:
  inspect-expression-case inspect-expression-cases$_{opt}$

inspect-expression-case:
  guarded-inspect-pattern `=>` *expression*

The identifiers in *inspect-pattern* are available in *expression*.

In the case that none of the patterns match the value, a `std::no_match` exception is thrown.

Inspect Pattern

guarded-inspect-pattern:
  inspect-pattern *guard*$_{opt}$

guard:
  `if (` *condition* `)`

inspect-pattern:
  `_`
  `nullptr`
  `*` *inspect-pattern*
  identifier (`@ (` *inspect-pattern* `)`)$_{opt}$
  alternative-selector inspect-pattern
  `(` *constant-expression* `)`
  constant-expression
  `[` *tuple-like-patterns*$_{opt}$ `]`

Wildcard pattern

inspect-pattern:
  `_`

The wildcard pattern matches any value and always succeeds.

`nullptr` pattern

inspect-pattern:
  `nullptr`

The `nullptr` pattern matches values `v` where `v == nullptr`.

Dereference pattern

inspect-pattern:
  `*` *inspect-pattern*

The dereferencing pattern matches values `v` where `*v` matches the nested pattern. Note that dereferencing the value occurs as part of the match.

Binding pattern

inspect-pattern:
  identifier (`@ (` *inspect-pattern* `)`)$_{opt}$

If `@` is not used, the binding pattern matches all values and binds the specified identifier to the value being matched. If `@` is used, the pattern is matched only if the nested pattern matches the value being matched.

Expression pattern

inspect-pattern:
  constant-expression
inspect-pattern:
  `(` *inspect-pattern* `)`

The expression pattern matches against all types supporting equality comparison. The pattern is valid if the matched type is the same as the *constant-expression* type. The pattern matches if the matched value is equal to the *constant-expression* value based on == comparison.

Note that the binding pattern takes precedence if there is ambiguity.

Tuple-like patterns

inspect-pattern:
  `[` *tuple-like-patterns*$_{opt}$ `]`

tuple-like-patterns:
  sequenced-patterns
  field-patterns

sequenced-patterns:
  inspect-pattern (`,` *sequenced-patterns*)$_{\text{opt}}$

field-patterns:
  field-pattern (`,` *field-patterns*)$_{\text{opt}}$

field-pattern:
  piece-selector `:` *inspect-pattern*

piece-selector:
  constant-expression
  identifier

Tuple-like patterns come in two varieties: a sequence of patterns and field patterns.

A sequenced pattern is valid if the following conditions are true:

1. The matched type is either a `class` with all public member variables or has a valid extract operator. Say the number of variables or arguments to extract is `n`.
2. There are exactly `n` patterns in the sequence.
3. Each of the sequenced patterns is valid for the corresponding piece in the matched value.

A field pattern is valid if the following conditions are true:

1. The matched type is either a `class` with all public member variables or has a valid extract operator.
2. *piece-selector*s, if they are *constant-expression*, must have the same type as the extract operator's `std::tuple_pieces` second template argument.
3. *piece-selector*s, if they are *identifier*s, must correspond to field names in the `class` with all public member variables.
4. Each of the field patterns is valid for the the corresponding piece in the matched value.

Both patterns match if the pattern for each piece matches its corresponding piece.

The *constant-expression* shall be a converted constant expression (5.20) of the type of the inspect condition's extract piece discriminator. The *identifier* will correspond to a field name if inspect's condition is an class or an identifier that is within scope of the class definition opting into the tuple-like pattern.

Design Choices

`inspect` as a statement and an expression

If `inspect` were a statement-only, it could be used in expressions via. a lambda function. For example:

```
const Vec3 opengl_color = [&c]{
  inspect(c) {
    red    => return Vec3(1.0, 0.0, 0.0)
    yellow => return Vec3(1.0, 1.0, 0.0)
    green  => return Vec3(0.0, 1.0, 0.0)
    blue   => return Vec3(0.0, 0.0, 1.0)
  } }();
```

Because we expect that `inspect` expressions will be the most common use case, we feel the syntactic overhead and tie-in to another complex feature (lambdas) too much to ask from users.

`inspect` with multiple arguments

It is a straightforward extension of the above syntax to allow for inspecting multiple values at the same time.

```
lvariant tree {
  int leaf;
  std::pair< std::unique_ptr<tree>, std::unique_ptr<tree> > branch;
}

bool sameStructure(const tree& lhs, const tree& rhs) {
  return inspect(lhs, rhs) {
           [leaf _, leaf _] => true
           [branch [*lhs_left, *lhs_right], branch [*rhs_left, *rhs_right]]
             =>    sameStructure(lhs_left , rhs_left)
                && samestructure(lhs_right, rhs_right)
           _ => false
         };
}
```

It is our intent that the final wording will allow for such constructions.

Open Design Questions

There is an open design questions that the authors would like feedback on from EWG.

Surface syntax

Herb Sutter suggested that extending the existing `switch/case` syntax would be more desirable than introducing a new `inspect/=>` syntax. For example, instead of ':', a switch statement could use braces.

```
switch (x) {
  case A { /*A*/ }
  case B { /*B*/ }
```

```
  default { }
}
```

`A` and `B` could, for example, be *inspect-pattern*s according to this proposal. The benefit would be that transitioning to a safer, more powerful switch is made easier. One drawback is that it may not be immediately obvious whether or not fallthrough is happening with a given switch statement.

Conclusion

We conclude that pattern matching enables programmers to express their solution in more intuitive, uniform and simple way. Concentrating attention on ‘what’ rather than ‘how’, allowing to express control flows in lightweight and readable way.

Acknowledgements

Thanks to Vicente Botet Escribá, John Skaller, Dave Abrahams, Bjarne Stroustrup, Bengt Gustafsson, and the C++ committee as a whole for productive design discussions. Also, Yuriy Solodkyy, Gabriel Dos Reis, and Bjarne Stroustrup’s prior research into generalized pattern matching as a C++ library has been very helpful. Also thanks to Herb Sutter for providing feedback on this proposal.

References

- V. Botet Escribá. Product types access. P0327R0. WG21
- V. Botet Escribá. Structured binding: customization points issues. P0326R0. WG21
- D. Sankel. C++ Langauge Support for Pattern Matching and Variants. davidsankel.com.
- Y. Solodkyy, G. Dos Reis, B. Stroustrup. Open Pattern Matching for C++. GPCE 2013.